

UpDown — Guía de despliegue a Mainnet

Versión: 2026-06-27 **Alcance:** llevar el producto completo (programa Solana de parimutuel pools + API + web + terminal de trading) de devnet a mainnet, con el detalle de SOL/USDC/ETH que hay que tener en cada wallet.

Convención de números: todos los costes de SOL salen de medidas reales del programa actual (rent-exempt calculado con `solana rent`, tamaño de cuentas del struct on-chain). El precio de SOL/ETH varía, así que doy cantidades en cripto y, donde pongo USD, es orientativo (multiplica por el precio del día).

1. Resumen de arquitectura

Qué se despliega y dónde:

Componente	Dónde corre	Qué necesita en mainnet
Programa Anchor <code>parimutuel_pools</code>	Solana mainnet-beta	deploy (rent del program data) + wallet upgrade authority
API (<code>apps/api</code>)	Railway (servicio api)	env mainnet, authority/treasury wallets, RPC mainnet de pago
Web (<code>apps/web</code>)	Railway (servicio web)	env <code>NEXT_PUBLIC_*</code> mainnet
Terminal (<code>apps/terminal</code>)	Railway (servicio terminal)	HyperLiquid mainnet , Privy prod, bridge mainnet
Postgres	Railway / Supabase	DB de producción con migraciones aplicadas

Cadenas implicadas: - **Solana mainnet:** pools, apuestas, vaults, USDC real. - **Arbitrum (EVM):** relayer del bridge + depósitos a HyperLiquid (gas en ETH). - **HyperLiquid mainnet:** trading real (lo fondea el usuario, no el operador).

2. Wallets y cuánto dinero necesitas (lo importante)

Hay 4 wallets de operador + USDC de liquidez. Recomendando **separarlas** (no reutilizar una sola) por seguridad y para no mezclar fees, igual que se hizo localhost/dev/prod.

2.1 Tabla de fondeo inicial

Wallet	Cadena	Para qué	Bloqueado	Colchón operativo	
Upgrade authority (deploy)	Solana	desplegar y poder actualizar el programa	~2.8–3.7 SOL (rent del program data)	~4 SOL para futuros upgrades (buffer transitorio)	8 SOL
Pool authority (<code>AUTHORITY_SECRET_KEY</code>)	Solana	crear/resolver/cerrar/claim; paga rent de Pool+vault y fees	rent flotante de pools abiertos (recuperable)	fees + margen	8 SOL
Treasury (<code>TREASURY_SECRET_KEY</code>)	Solana	fondear el liquidity bot + sus fees	—	fees	2 SOL + USDC de liquidez
Bridge relayer (<code>BRIDGE_RELAYER_PRIVATE_KEY</code>)	Arbitrum	firmar depósitos a HyperLiquid	—	gas en ETH	0.03–0.05 ETH
Liquidez (USDC)	Solana	stakes del bot para dar volumen visible	—	recuperable según resuelven los pools	tú decides (ej. \$200–\$1000)

Total mínimo para operar cómodo: ~18 SOL + ~0.05 ETH + tu presupuesto de USDC. (Si juntas upgrade authority y pool authority en una sola wallet, baja a ~12 SOL, pero no lo recomiendo: la upgrade authority debería ser fría/multisig.)

2.2 De dónde salen los números de SOL (rent real del programa actual)

Medido con `solana rent` sobre el tamaño real de las cuentas:

Cuenta	Tamaño	Rent-exempt
Pool (<code>8 + Pool::INIT_SPACE = 266 B</code>)	266 bytes	0.00274224 SOL
Vault (SPL token account)	165 bytes	0.00203928 SOL
UserBet (apuesta, <code>8 + 99 = 99 B</code>)	99 bytes	0.00157992 SOL
Tournament (~150 B)	150 bytes	0.00193488 SOL
Program data (deploy fresco ~400 KB)	~400 KB	~2.78 SOL

Derivados: - **Crear un pool** = Pool + vault = **0.00478152 SOL** (lo paga la pool authority, se **recupera entero** al cerrar el pool con `close_pool`). - **Una apuesta** = **0.00157992 SOL** (lo paga el bettor o el relayer gasless; se recupera al hacer claim o `close_losing_bet`).

2.3 Cómo dimensionar el colchón de la pool authority

El rent de los pools es recuperable, pero mientras están abiertos queda inmovilizado. Ejemplo con 200 pools abiertos a la vez (cripto horario + deportes + Polymarket):

```
200 pools × 0.00478 SOL = 0.96 SOL inmovilizados (vuelven al cerrar)
fees: ~3 tx por pool (create/resolve/close) a 0.000005 SOL ≈ despreciable
```

Con **5–8 SOL** la pool authority aguanta cientos de pools simultáneos y meses de fees. Pongo 8 SOL recomendado para tener margen sin estar recargando.

2.4 Coste del deploy del programa

- El program data account queda con **~2.78 SOL** (programa actual ~197 KB, el loader reserva ~2x para permitir upgrades). En dev quedó en 3.67 SOL con un build mayor, así que cuenta con **3–4 SOL bloqueados**.
- Durante el deploy se crea un **buffer temporal** del mismo tamaño que se cierra al terminar, así que necesitas ~2x disponible en el momento. Por eso **8 SOL** en la wallet de deploy (queda ~3–4 bloqueados + margen para reintentos y futuros upgrades).

3. Prerrequisitos

- `solana-cli` y `anchor-cli` (en este proyecto: solana 3.0.x / anchor 0.31.1, vía WSL).
- Acceso a Railway (entorno de producción/mainnet) y a la DB de prod.
- Un **RPC de mainnet de pago** (Helius/QuickNode/Triton). El RPC público de mainnet no aguanta `getProgramAccounts` ni el ritmo de creación de pools.
- USDC real en Solana para la liquidez.
- (Muy recomendado) **auditoría del programa** antes de mover fondos reales, y poner la upgrade authority en un **multisig (Squads)** o dejar el programa inmutable tras auditar.

4. Paso a paso

Paso 0 — Generar y respaldar las wallets de mainnet

No reutilices las de devnet. Genera nuevas:

```
solana-keygen new -o ~/keys/mainnet-upgrade-authority.json # deploy/upgrade
solana-keygen new -o ~/keys/mainnet-pool-authority.json # AUTHORITY_SECRET_KEY
solana-keygen new -o ~/keys/mainnet-treasury.json # TREASURY_SECRET_KEY
```

Respalda los 3 archivos en tu gestor de contraseñas + copia fría. La del relayer EVM créala en una wallet EVM (MetaMask/Argent) y exporta su private key.

Paso 1 — Fondear las wallets

- Envía SOL a upgrade authority (8) y pool authority (8), treasury (2).
- Envía ~0.05 ETH (en Arbitrum) al relayer.
- Compra/transfiere el USDC de liquidez a la treasury (USDC mint mainnet: `EPjFWdd5AufqSSqm2qN1xzybapC8G4wEGGkZwyTDt1v`).

Verifica: `solana balance <pubkey> --url mainnet-beta`.

Paso 2 — Generar el keypair del programa mainnet

```
solana-keygen new -o programs/parimutuel_pools-MAINNET-keypair.json # gitignored
solana-keygen pubkey programs/parimutuel_pools-MAINNET-keypair.json # = nuevo PROGRAM_ID mainnet
```

Apunta ese pubkey: es tu `PROGRAM_ID` de mainnet.

Paso 3 — Build y deploy a mainnet

Igual que el flujo dev/local, pero contra mainnet y con la upgrade authority:

1. Pon el `declare_id!` (en `programs/parimutuel_pools/src/lib.rs`) y `Anchor.toml` `[programs.mainnet]` al `PROGRAM_ID` nuevo de mainnet.
2. `anchor build`
3. Deploy con la wallet de upgrade authority como fee-payer: `bash solana program deploy target/deploy/parimutuel_pools.so \ --program-id programs/parimutuel_pools-MAINNET-keypair.json \ --keypair ~/keys/mainnet-upgrade-authority.json \ -url mainnet-beta`
4. **Revierte** `declare_id!` y `Anchor.toml` a su valor anterior (no commitear el cambio temporal de id), igual que se hace con dev/local.
5. Verifica: `solana program show <PROGRAM_ID> --url mainnet-beta` (Authority = tu upgrade authority, Balance ≈ 3 SOL).

Paso 4 — Configurar env vars (mainnet)

Servicio api (mainnet):

```
PROGRAM_ID=<program id mainnet>
AUTHORITY_SECRET_KEY=<array de mainnet-pool-authority.json>
TREASURY_SECRET_KEY=<array de mainnet-treasury.json>
SOLANA_CLUSTER=mainnet
SOLANA_RPC_URLS=<tu RPC mainnet de pago, coma-separado para failover>
USDC_MINT=EPjFWdd5AufqSSqm2qN1xzybapC8G4wEGGkZwyTDt1v
CORS_ORIGIN=https://updown.my
WEB_BASE_URL=https://updown.my
EXCHANGE_KEY_ENCRYPTION_SECRET=<secreto propio de prod>
BRIDGE_HL_TESTNET=false
BRIDGE_RELAYER_PRIVATE_KEY=<relayer EVM>
ARBITRUM_RPC_URL=<rpc arbitrum mainnet>
# Side-effects: enciéndelos solo cuando quieras
LIQUIDITY_BOT_KEYS=<claves del bot, o vacío para no correrlo>
X_API_KEY / X_API_SECRET / X_ACCESS_TOKEN / X_ACCESS_SECRET=<si quieres el poster real>
TESTING_MODE= (vacío/off en mainnet)
```

Servicio web (mainnet):

```

NEXT_PUBLIC_PROGRAM_ID=<program id mainnet>
NEXT_PUBLIC_API_URL=https://<api mainnet>
NEXT_PUBLIC_USDC_MINT=EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v
NEXT_PUBLIC_SOLANA_RPC_URL=<rpc mainnet>
NEXT_PUBLIC_ENV=production
NEXT_PUBLIC_TERMINAL_URL=https://<terminal mainnet>
NEXT_PUBLIC_PRIVY_APP_ID / NEXT_PUBLIC_PRIVY_CLIENT_ID=<app Privy de prod>

```

Servicio terminal (mainnet):

```

NEXT_PUBLIC_HYPERLIQUID_TESTNET=false
NEXT_PUBLIC_HYPERLIQUID_API_URL=https://api.hyperliquid.xyz
NEXT_PUBLIC_API_URL=https://<api mainnet>
NEXT_PUBLIC_APP_URL=https://updown.my
NEXT_PUBLIC_ARBITRUM_RPC_URL / NEXT_PUBLIC_SOLANA_MAINNET_RPC_URL=<rpc mainnet>
NEXT_PUBLIC_PRIVY_APP_ID / NEXT_PUBLIC_PRIVY_CLIENT_ID=<app Privy de prod>
NEXT_PUBLIC_HYPERLIQUID_BUILDER_ADDRESS / MAX_FEE=<tu builder address + fee>

```

CRÍTICO: `NEXT_PUBLIC_HYPERLIQUID_TESTNET=false` y el API url de HL mainnet. Si se queda el default, `lib/exchange.ts` ya resuelve a mainnet, pero déjalo explícito.

Paso 5 — Base de datos

- DB de producción **limpia** (sin pools de test). Si reusas una DB, wipea los datos (conservando config) antes de abrir al público.
- Migraciones: el `start` del api corre `prisma migrate deploy` al arrancar. Verifica que aplica todo contra la DB de prod.

Paso 6 — USDC y liquidity bot (opcional)

- Manda el USDC de liquidez a la treasury.
- Configura el bot desde el admin (tab Liquidity): caps por pool, kill switch, varianza. Empieza con un presupuesto pequeño; los stakes vuelven según resuelven los pools (menos lo que pierda contra bettors reales).

Paso 7 — HyperLiquid + bridge mainnet

- HL mainnet: el trading lo fondea el usuario (deposita su propio USDC a HL). El operador no pone capital de trading.
- Bridge a HL: el relayer (Arbitrum) paga el gas de los depósitos. Mantén ~0.03–0.05 ETH.
- Builder fee: si cobras builder fee en HL, configura `HYPERLIQUID_BUILDER_ADDRESS` + `HYPERLIQUID_BUILDER_FEE`.

Paso 8 — Desplegar servicios y verificar

1. Apunta el entorno mainnet de Railway a la rama de producción y redeploy los 3 servicios.
2. Checklist de verificación:
3. `solana program show <PROGRAM_ID> --url mainnet-beta` → ejecutable, authority correcta.
4. Crear 1 pool de prueba → confirmar que la cuenta on-chain es owner = tu PROGRAM_ID y authority = tu pool authority (mismo método que usamos en dev).
5. Una apuesta pequeña de prueba con USDC real → claim → cerrar pool → confirmar que el rent vuelve a la pool authority.
6. Web carga pools, sin errores de CORS.
7. Terminal: una orden mínima en HL mainnet, TP/SL, y cerrar.
8. Notificaciones llegan y persisten en DB.

4.bis Scripts y comandos (copia y pega)

Todo desde la raíz del repo, en WSL (donde viven `solana` / `anchor`).

A. Generar y respaldar wallets

```
solana-keygen new -o ~/keys/mainnet-upgrade-authority.json # deploy/upgrade
solana-keygen new -o ~/keys/mainnet-pool-authority.json # AUTHORITY_SECRET_KEY
solana-keygen new -o ~/keys/mainnet-treasury.json # TREASURY_SECRET_KEY
solana-keygen new -o programs/parimutuel_pools-MAINNET-keypair.json # PROGRAM_ID (gitignored)

# pubkeys (apúntalas)
solana-keygen pubkey ~/keys/mainnet-upgrade-authority.json
solana-keygen pubkey ~/keys/mainnet-pool-authority.json
solana-keygen pubkey ~/keys/mainnet-treasury.json
solana-keygen pubkey programs/parimutuel_pools-MAINNET-keypair.json # = PROGRAM_ID mainnet
```

B. Verificar fondeo antes de desplegar

```
solana balance <upgrade-authority-pubkey> --url mainnet-beta # >= ~8 SOL
solana balance <pool-authority-pubkey> --url mainnet-beta # >= ~8 SOL
solana balance <treasury-pubkey> --url mainnet-beta # >= ~2 SOL
```

C. Convertir una private key base58 (Phantom) a keypair JSON

Para meter `AUTHORITY_SECRET_KEY` / `TREASURY_SECRET_KEY` en Railway necesitas el array de 64 bytes. Si tienes la clave en base58:

```
node -e 'const b58=process.argv[1];const A="123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz";const m={};[...A
```

D. Build + deploy del programa (script incluido)

```
./scripts/deploy-program.sh \
  programs/parimutuel_pools-MAINNET-keypair.json \
  mainnet-beta \
  ~/keys/mainnet-upgrade-authority.json
```

El script pone el `declare_id!` al id nuevo, hace `anchor build`, despliega con `solana program deploy`, y **revierte** el id con `git checkout` (deja el source intacto). Pide confirmación y muestra el balance del payer antes de gastar.

Equivalente manual (si prefieres a mano):

```
# 1. editar declare_id! en programs/parimutuel_pools/src/lib.rs y Anchor.toml -> PROGRAM_ID mainnet
anchor build
solana program deploy target/deploy/parimutuel_pools.so \
  --program-id programs/parimutuel_pools-MAINNET-keypair.json \
  --keypair ~/keys/mainnet-upgrade-authority.json \
  --url mainnet-beta
git checkout -- programs/parimutuel_pools/src/lib.rs Anchor.toml # revertir
solana program show <PROGRAM_ID> --url mainnet-beta # verificar
```

E. Base de datos (migraciones / wipe)

```
# Migraciones (el `start` del api ya corre prisma migrate deploy; manual:)
DATABASE_URL="<prod>" DIRECT_URL="<prod>" pnpm --filter api prisma migrate deploy

# Wipe de datos conservando config (SOLO si reusas una DB con datos de test)
DATABASE_URL="<prod>" node apps/api/scripts/wipe-all-data.mjs
```

F. Env vars en Railway por CLI (alternativa al dashboard)

```
railway login
railway link          # elige proyecto + entorno mainnet
railway service       # elige el servicio (api / web / terminal)
railway variables --set PROGRAM_ID=<id> --set SOLANA_CLUSTER=mainnet ...
```

G. Verificación post-deploy

```
# El programa quedó ejecutable y con tu upgrade authority
solana program show <PROGRAM_ID> --url mainnet-beta

# Rent de referencia (sanity check de tamaños)
solana rent 266 # Pool -> 0.00274224 SOL
solana rent 165 # Vault -> 0.00203928 SOL
solana rent 99 # UserBet-> 0.00157992 SOL

# Tras crear 1 pool de prueba: confirmar que su cuenta on-chain pertenece a TU
# programa y authority (igual que verificamos dev). Sustituye <POOL_ID> y <RPC>:
node -e 'const {Connection,PublicKey}=require("@solana/web3.js");(async()=>{const c=new Connection(process.argv[2]);const
```

5. Costes recurrentes (operación)

Concepto	Coste	Notas
Rent de pools abiertos	~0.00478 SOL/pool, recuperable	vuelve al cerrar
Fees de tx (create/resolve/close/claim)	~0.000005 SOL/firma	despreciable incluso a gran volumen
RPC mainnet	mensualidad del proveedor	el mayor coste fijo real
Gas del relayer (Arbitrum)	ETH por depósito a HL	recarga cuando baja
Liquidez del bot	USDC, recuperable	opcional
HL trading	lo pone el usuario	el operador no fondea

La pool authority recupera rent al cerrar; vigila su balance y recárgala si baja de ~2 SOL.

6. Seguridad y rollback

- **Auditar** el programa antes de fondos reales. Maneja dinero de usuarios.
- Upgrade authority en **multisig (Squads)** o programa **immutable** tras auditar.
- Backups de los 4 keypairs en frío. Sin ellos no puedes operar ni actualizar.
- Rollback: si algo falla tras un upgrade, redeploy de la versión anterior del **.so** con la upgrade authority. Los pools existentes siguen on-chain.
- Mantén devnet (dev/localhost) como entorno de pruebas; nunca pruebes en mainnet primero.

7. Apéndice — Resumen de cantidades

```
Upgrade authority (Solana):  8 SOL  (~3 bloqueados en el programa + margen)
Pool authority   (Solana):  8 SOL  (rent flotante recuperable + fees)
```

```
Treasury      (Solana):    2 SOL    + USDC de liquidez
Bridge relay  (Arbitrum): 0.03~0.05 ETH
Liquidez      (USDC):    a tu criterio (ej. 200-1000 USD)
-----
TOTAL para arrancar: ~18 SOL + ~0.05 ETH + USDC de liquidez
```

USDC mint mainnet (Solana): [EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v](#) HyperLiquid mainnet API:
<https://api.hyperliquid.xyz>